Veritas: A Lean-Backed Pre-Trade Verifier for Agent-Native Finance

The Veritas Project
github.com/lizjq66/Veritas

April 2026

Abstract

Automated trading agents increasingly act on capital through exchange APIs at rates that outpace manual oversight. The public discussion frames this as a *prediction quality* problem and invests accordingly in larger models and denser backtests. We argue that the more load-bearing failure mode of current agent stacks is *pre-trade approval*: agents take trades whose operators would have intervened had they been given the chance, and do so without producing a legible account of what they are about to do.

We present **Veritas**, a pre-trade verification system with a small fixed contract. Given a trade proposal, account constraints, and existing portfolio, Veritas returns a structured certificate indicating APPROVE / RESIZE / REJECT for each of three orthogonal gates: (1) signal consistency, (2) strategy-constraint compatibility, (3) portfolio interference. Gate logic is written in Lean 4 [1] with closed soundness proofs; Python is transport. Veritas is not a trading agent; it is the thing agents call before moving capital. This paper describes the design, the five gate-layer soundness theorems, the 20-axiom Float basis the proofs depend on, and the v0.1 deliverable.

1 Introduction

The first visible failure of automated trading agents is that they lose money. The first visible fix, therefore, is that they should predict better. Every LLM-trading-agent release ships with a benchmark bar chart.

Empirically, though, the first several generations of agent stacks fail in a recognizable second way: they take trades their operators would have blocked if they had been given the chance. An agent decides to lever up a correlated basket one minute before a scheduled Fed release. An agent opens a ten-percent notional position on a pair whose liquidity it has not traded before. An agent doubles down on a rejected thesis because the reviewer prompt does not surface the reject.

These are not prediction failures. They are *approval* failures. The agent knew what it wanted to do; nothing in the system forced it to articulate the claim, check the claim against an envelope, and account for existing exposure before committing.

The industry response so far has been either (a) add another LLM to critique the first, or (b) bolt post-trade risk controls on top of an opaque decision loop. The first restates the problem in a larger system; the second is reactive by construction — it learns about a breach after the money has moved.

Between intent and execution, there should be a *pre-trade* gate that is (i) narrower than the agent, (ii) legibly auditable, and (iii) not produced by the same process that produced the trade. This is the space Veritas occupies.

Thesis. We argue that *pre-trade verification*, not prediction quality, is the load-bearing safety property of agent-native trading stacks, and that the right instantiation is a narrow verifier with a formally-verified kernel — not a better trader with safety bolted on. The remainder of this paper develops this claim and reports on a working v0.1 instantiation.

Contributions.

- A three-gate decomposition of pre-trade verification (Section 3).
- A Lean 4 implementation with five gate-layer soundness theorems, two Kelly theorems, five position-sizing theorems, an exit-exhaustiveness theorem, and two reliability-update theorems, all with closed proofs (Section 5).
- A uniform HTTP / MCP [3] / Python / CLI contract over a single kernel, covered by 86 tests with no mocks on the kernel.
- CI-enforced boundary invariants preventing the Python layer from reintroducing decision logic or minting approvals outside the kernel.

2 The Verifier Boundary

The formal-methods community has decades of experience with tools that sit between a producer and a consumer and discharge obligations on the producer's behalf: model checkers, static analyzers, type systems, SMT solvers. The design pattern is consistent: small, fixed input contract; small, structured output; no shared state with the caller; decision kernel isolated from I/O.

A trading-agent verifier deserves the same shape. We adopt it directly:

- **Small fixed contract.** A `TradeProposal`, an `AccountConstraints`, a `Portfolio`. Every field is a simple value. No callbacks, no futures, no handles.
- **Small structured output.** A `Certificate` of three per-gate verdicts (APPROVE / RESIZE / REJECT) plus reason codes and attached assumptions. No exceptions.
- **No shared state.** Veritas is a pure function: no session, no token, no subscription.
- **Isolated kernel.** Gate logic lives in a Lean 4 kernel compiled to a native binary. The Python adapters that speak HTTP, MCP, and SQLite contain no gate decisions.

A verifier with this shape composes well with existing agents. A caller can adopt it by adding one call per proposal, with no architectural change. The verifier does not require access to the caller's alpha, model weights, or prompts; it only needs the output of the decision the caller is about to make.

3 The Three Gates

We decompose the approval obligation into three orthogonal questions. Each is a gate returning one of APPROVE, RESIZE, or REJECT.

3.1 Gate 1 — Signal Consistency

Given the proposed trade and the market context the caller claims to be acting under, does the proposal cohere? Gate 1 dispatches over a *policy registry* — a list of named strategies, each with

its own decider and assumption extractor. For every submitted proposal, Gate 1 runs every registered strategy's decider on the context, collects the set of firing signals, and checks that (i) at least one strategy fires, (ii) all firing strategies agree on direction (`MutuallyConsistent`), (iii) the proposal's direction matches them, and (iv) the union of attached assumptions is non-empty. On approval Gate 1 returns this union, so downstream auditors see every bet the caller is implicitly committing to.

The non-triviality of Gate 1 depends on the registry containing at least two strategies that can meaningfully disagree. The current registry ships with `funding_reversion` and `basis_reversion`, which produce opposite-direction signals on the same market context under conditions of long-side pressure (positive funding rate plus perp-rich-vs-spot). That disagreement is the case Gate 1's consistency check is designed to detect and reject.

3.2 Gate 2 — Strategy-Constraint Compatibility

Given the caller's account envelope (equity, reliability of the attached assumption, sample size, leverage cap, stop-loss), is the requested notional inside the ceiling? Gate 2 returns the ceiling when the caller is over it (RESIZE), rather than silently approving at the larger number. Gate 2 rejects when no non-zero size is admissible — below the reliability threshold, or with non-positive leverage. This is the gate whose formal content is strongest in v0.1.

3.3 Gate 3 — Portfolio Interference

Given the caller's existing positions and the (possibly resized) proposal, does the combined exposure violate portfolio-wide constraints? The current version catches opposite-direction conflicts and enforces a gross-notional cap; richer correlation handling across asset classes becomes meaningful once the registry contains strategies over heterogeneous assets.

The gates execute in order. Gate 2 sees the original proposal; Gate 3 sees the Gate 2 output. If any gate rejects, downstream gates receive an `upstream_gate_rejected` marker and the final notional is zero.

Gate	Question	Output
1. Signal consistency	Direction agrees with policy?	approve / reject
2. Constraint compat.	Size fits the reliability-adjusted ceiling?	approve / resize / reject
3. Portfolio interference	Clash with existing positions or cap breach?	approve / resize / reject

Table 1: The three gates.

4 Lean-Backed Kernel

The design leans on Lean 4 for the kernel. Two properties matter.

Compile-checked API surface. Veritas encodes properties the type system would otherwise leave as conventions. Exit reasons are a sum type with three constructors (`AssumptionMet`, `AssumptionBroke`, `StopLoss`); `ReliabilityStats` carries `wins` $\leq$ `total` at the type level, so the constructor cannot be called with inconsistent counts; `Verdict` has three tags, each carrying the data needed for that tag and nothing more.

Theorem-backed numeric behavior. `Finance.PositionSizing` ships five theorems: non-negativity, a 25% equity cap, zero-at-no-edge, monotonicity in reliability, and an exploration cap at 1%. `Strategy.ExitLogic` ships `exitReason_exhaustive`. `Learning.Reliability` ships

bounded-in-$[0, 1]$ and monotone-on-wins. These eliminate categories of silent miscount; a reviewer has a small set of facts about which to argue.

Cost. Lean 4's `Float` is an opaque FFI type with no algebraic library. Theorems depend on 20 axioms [4] capturing IEEE 754 ordering and arithmetic. Thirteen are exact; seven are rounding-dependent within the numeric ranges Veritas uses. Reducing the rounding-dependent axioms — ideally to zero by migrating policy arithmetic to exact types (`Rat` or Mathlib's `Real` [2]) — is the chief v0.2 technical goal. The axiom set is disclosed up front; hiding it would be a trust regression.

5 Gate-Layer Soundness Theorems

Each gate file exports a soundness theorem stating what an APPROVE (or RESIZE) verdict from that gate *means*, independent of the underlying strategy and finance modules. The theorems, all closed under the 20 Float axioms, are summarized in Table 2.

Theorem	Statement (informal)
`verifySignal_approve_ implies_consistent`	Gate 1 approve $\Rightarrow$ the proposal is signal-consistent.
`checkConstraints_approve_ within_ceiling`	Gate 2 approve $\Rightarrow$ p.`notional` $\leq$ `ceiling`.
`checkConstraints_resize_ respects_ceiling`	Gate 2 resize $n \Rightarrow n \leq$ `ceiling`.
`checkPortfolio_approve_ respects_cap`	Gate 3 approve $\Rightarrow$ gross notional stays under the cap.
`certificate_soundness`	`approves` true $\Rightarrow$ Gate 1 consistent and Gate 2/3 non-reject.

Table 2: Gate-layer soundness theorems in `Veritas/Gates/`.

As an example, Gate 2's ceiling theorem states:
Theorem 1 (`checkConstraints_approve_within_ceiling`)**.** *For every proposal p and account constraints c, if* `checkConstraints`$(p, c) =$ APPROVE, *then*

$$p.\texttt{notionalUsd} \leq \texttt{calculatePositionSize}(c.\texttt{equity}, c.\texttt{reliability}, c.\texttt{sampleSize}).$$

The right-hand side is in turn bounded by the five `positionSize_*` theorems: non-negative, $\leq 0.25 \cdot c.\texttt{equity}$, zero when reliability ≤ 0.5 post-exploration, monotone in reliability, fixed at $0.01 \cdot c.\texttt{equity}$ during the first ten samples.

The combined soundness theorem connects the chain:
Theorem 2 (`certificate_soundness`)**.** *For every proposal p, account constraints c, and portfolio port, let* `cert` $:=$ `emitCertificate`$(p, c, port)$. *If* `cert.approves` $=$ *true, then*

1. `signalConsistent`(p) *holds;*
2. *the certificate's* `gate2` *verdict is not a rejection;*
3. *the certificate's* `gate3` *verdict is not a rejection.*

Numeric bounds for Gates 2 and 3 follow by instantiating the per-gate theorems on the certificate's actual verdicts. The proof of `certificate_soundness` proceeds by case analysis on the `verifySignal` output and the intermediate `g2` verdict; the unreachable RESIZE case for Gate 1 is discharged by exhaustive case analysis on the body of `verifySignal`, which produces only APPROVE or REJECT.

Python cannot bypass the kernel. The Python layer is constrained by three CI-enforced invariants:

1. No Python file branches on Veritas decision types (`Signal`, `ExitDecision`, `PositionSize`).
2. No Python file constructs `Verdict` values outside `python/schemas.py` (the dataclass) or `python/verifier.py` (deserialization from the kernel).
3. No Python file invokes `veritas-core` via `subprocess` outside `python/bridge.py`.

These are checked by `tests/test_bypass_invariant.py` and `tests/test_loop.py`.

6 Claim Discipline

We want to be explicit about the scope of what Veritas publishes.

What Veritas verifies. That the approved notional returned by the Lean kernel satisfies the theorems of Section 5. That every exit classified by `Strategy.ExitLogic` is one of three exhaustive reasons. That reliability updates stay in $[0, 1]$ and are monotone on wins. That each gate in `Veritas/Gates/` executes the pure function its specification describes.

What Veritas does not verify.

- The correctness of the caller's alpha. An adversarial proposal with a fabricated funding rate is checked against policy but not against a ground-truth data feed.
- The correctness of the bundled funding-reversion policy. Veritas verifies that the policy is applied consistently; it does not claim the policy is profitable.
- Anything about a specific execution venue. After the certificate is issued, what the caller does is the caller's problem.
- Float-level numerical edge cases outside Veritas's numeric ranges (probabilities in $[0, 1]$, Kelly fractions, small `Nat` counts).
- Correctness of the Python transport. An adapter bug could fail to deliver a certificate; no theorem prevents that. Callers are expected to treat missing certificates as rejections.

7 Related Work

Institutional trading desks routinely run pre-trade risk checks; the novelty here is not the category, it is the combination of (i) an agent-native contract, (ii) a Lean-backed kernel, and (iii) public, legible certificates with machine-readable reason codes.

Formal-methods work in finance has historically concentrated on smart-contract verification. Tools such as Certora [5] and the Move Prover [6] formalize on-chain logic. Veritas formalizes *off-chain policy* — the step that decides whether an order should be sent at all — and leaves the on-chain side to existing tools. Mathlib [2] provides the broader formal mathematics library on which future versions of Veritas are expected to build when migrating to exact arithmetic.

Agent-evaluation work — benchmarks such as AgentBench [7], the Inspect framework for red-teaming agents, and domain-specific suites for trading agents — focuses on pre-deployment screening. Veritas is orthogonal: it is a run-time gate, not a pre-deployment evaluation.

The Model Context Protocol [3] provides a uniform way for agents to call tools; Veritas ships an MCP tool (`verify_proposal`) alongside the HTTP and Python surfaces.

8 Implementation

v0.1 deliverables. The `Verifier` Python surface, the `POST /verify/*` HTTP endpoints, the MCP `verify_proposal` tool, and the `veritas-core` CLI all produce the same certificate for the same input, enforced by 86 passing tests with no mocks on the kernel. `lake build` produces the compiled kernel binary with zero `sorry`. The 20 Float axioms are classified (13 exact, 7 rounding-dependent) in `Veritas/Finance/FloatAxioms.lean`.

Example runner. The repository ships a funding-reversion trading agent that calls Veritas on every tick. It is a demonstration of a caller, not the product. Its Hyperliquid adapters (`HyperliquidObserver`, `HyperliquidExecutor`) are reference integrations; exercising them against live testnet traffic is not a v0.1 ship requirement. The verifier's correctness against fixture proposals is.

Interactive playground. A browser-visible playground at `/` renders the three-gate verdicts visually and fires the `POST /verify/proposal` round-trip on six preset scenarios (clean approve, direction conflict, no signal under policy, oversize → resize, no edge, portfolio conflict). It exists for human evaluators; agents bypass it and use the HTTP contract directly.

9 Conclusion

The question a trading agent should answer before touching capital is not *"what do you want to do?"* but *"what do you want to do, under what assumption, inside what envelope, against what you already hold, and why?"* Veritas exists because nothing else in the agent-native stack asks that question in a way the caller can be forced through and the reviewer can audit. We would rather have a small, legible gate than a large, opaque trader. The v0.1 instantiation is the smallest such gate we could ship while keeping all three questions on the critical path, and its formal and engineering scope is the evidence that the thesis of Section 1 is buildable, not merely arguable.

References

[1] Leonardo de Moura and Sebastian Ullrich. *The Lean 4 Theorem Prover and Programming Language*. Proceedings of CADE 28, 2021.

[2] The Mathlib Community. *The Lean Mathematical Library*. Proceedings of CPP 2020, ACM, 2020.

[3] Anthropic. *Model Context Protocol Specification*, 2024. `https://modelcontextprotocol.io`

[4] The Veritas Project. *Float Axioms: IEEE 754 properties formalized for the Veritas numeric range*, v0.1, April 2026. See `Veritas/Finance/FloatAxioms.lean`.

[5] Certora. *The Certora Prover: formal verification for smart contracts*. `https://www.certora.com`

[6] David Dill, Wolfgang Grieskamp, Junkil Park, Shaz Qadeer, Meng Xu, Emma Zhong. *Fast and Reliable Formal Verification of Smart Contracts with the Move Prover*. TACAS 2022.

[7] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, Jie Tang. *AgentBench: Evaluating LLMs as Agents*. ICLR 2024.